

UNIT – II:

Operators and Expressions: Types - Operators- Arithmetic Operators, Comparison (Relational) Operators, Assignment Operators, Logical Operators, Bitwise Operators, Membership Operators, Identity Operators, Expressions and order of evaluations

Control Flow: if, if-elif-else, for, while, break, continue, pass

Operators:

Operators are special symbols in Python that carry out arithmetic or logical computation. The value that the operator operates on is called the operand.

For example:

```
>>>2+3
```

```
5
```

Here, + is the operator that performs addition. 2 and 3 are the operands and 5 is the output of the operation.

Arithmetic operators

Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication, modular division etc.

Operator	Meaning	Example
+	Add two operands or unary plus	x + y+ 2
-	Subtract right operand from the left or unary minus	x - y- 2
*	Multiply two operands	x * y
/	Divide left operand by the right one (always results into float)	x / y
%	Modulus - remainder of the division of left operand by the right	x % y (remainder of x/y)
//	Floor division - division that results into whole number adjusted to the left in the number line	x // y
**	Exponent - left operand raised to the power of right	x**y (x to the power y)

Example : Arithmetic operators in Python

```
x=15
y=4
print('x+y=',x+y)
print('x-y=',x-y)
print('x*y=',x*y)
print('x/y=',x/y)
print('x//y=',x//y)
print('x**y=',x**y)
```

Output

```
x+y=19
x-y=11
x*y=60
x/y=3.75
x//y=3
x**y=50625
```

Comparison (Relational) operators

Comparison operators are used to compare values. It returns either True or False according to the condition.

Operator	Meaning	Example
>	Greater than - True if left operand is greater than the right	x > y
<	Less than - True if left operand is less than the right	x < y
==	Equal to - True if both operands are equal	x == y
!=	Not equal to - True if operands are not equal	x != y
>=	Greater than or equal to - True if left operand is greater than or equal to the right	x >= y
<=	Less than or equal to - True if left operand is less than or equal to the right	x <= y

Example : Comparison operators in Python

```
x = 10
y = 12
print('x > y is',x>y)
print('x < y is',x<y)
print('x == y is',x==y)
print('x != y is',x!=y)
print('x >= y is',x>=y)
print('x <= y is',x<=y)
```

Output

```
x > y is False
x < y is True
x == y is False
x != y is True
x >= y is False
x <= y is True
```

Assignment operators

- Assignment operators are used in Python to assign values to variables.
- a = 5 is a simple assignment operator that assigns the value 5 on the right to the variable a on the left.
- There are various compound operators in Python like a += 5 that adds to the variable and later assigns the same. It is equivalent to a = a + 5.

Operator	Example	Equivalent to
=	x = 5	x = 5
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5
//=	x //= 5	x = x // 5
**=	x **= 5	x = x ** 5
&=	x &= 5	x = x & 5
=	x = 5	x = x 5
^=	x ^= 5	x = x ^ 5
>>=	x >>= 5	x = x >> 5
<<=	x <<= 5	x = x << 5

Logical operators

- There are three logical operators: **and**, **or**, and **not**.
- The semantics (meaning) of these operators is similar to their meaning in English. For example, $x > 0$ and $x < 10$ is true only if x is greater than 0 and less than 10.
- $n \% 2 == 0$ or $n \% 3 == 0$ is true if either (or) both of the conditions is true, that is, if the number is divisible by 2 or 3.
- Finally, the not operator negates a boolean expression, so $\text{not}(x)$ is true if x is false.

Operator	Meaning	Example
And	True if both the operands are true	x and y
Or	True if either of the operands is true	x or y
Not	True if operand is false (complements the operand)	not x

Example : Logical Operators in Python

```
x = True
y = False
print('x and y is',x and y)
print('x or y is',x or y)
print('not x is',not x)
```

Output

```
x and y is False
x or y is True
not x is False
```

Bitwise operators

- Bitwise operators act on operands as if they were strings of binary digits.
- They operate bit by bit, hence the name.
- *For example*, 2 is 010 in binary and 7 is 111.
- In the table below: Let $x = 10$ (0000 1010 in binary) and $y = 4$ (0000 0100 in binary)

Operator	Meaning	Example
&	Bitwise AND	$x \& y = 0$ (0000 0000)
	Bitwise OR	$x y = 14$ (0000 1110)
~	Bitwise NOT	$\sim x = -11$ (1111 0101)
^	Bitwise XOR	$x \wedge y = 14$ (0000 1110)
>>	Bitwise right shift	$x >> 2 = 2$ (0000 0010)
<<	Bitwise left shift	$x << 2 = 40$ (0010 1000)

Membership operators

- These are used to test whether a value or variable is found in a sequence (string, list, tuple, set and dictionary).
- There are two membership operators in python: **in** and **not in**.
- In a dictionary we can only test for presence of key, not the value.

Operator	Meaning	Example
In	True if value/variable is found in the sequence	5 in x
not in	True if value/variable is not found in the sequence	5 not in x

Example: Membership operators in Python

```
x = 'Hello world'
```

```
y = {1:'a',2:'b'}
```

```
print('H' in x)
```

```
print('hello' not in x)
```

```
print(1 in y)
```

```
print('a' in y)
```

Output

True

True

True

False

Here, 'H' is in x but 'hello' is not present in x (remember, Python is case sensitive). Similarly, 1 is key and 'a' is the value in dictionary y. Hence, 'a' in y returns False.

Identity operators

- These are used to check if two values (or variables) are located on the same part of the memory.
- There are two identity operators in python: **is** and **is not**.
- Two variables that are equal does not imply that they are identical.

Operator	Meaning	Example
Is	True if the operands are identical (refer to the same object)	x is True
is not	True if the operands are not identical (do not refer to the same object)	x is not True

Example : Identity operators in Python

```
x1 = 5
```

```
y1 = 5
```

```
print(x1 is not y1)
```

```
print(x1 is y1)
```

Output

False

True

Operator Precedence

- When more than one operator appears in an **expression**, the order of evaluation depends on the **rules of precedence**.
- Python follows the same precedence rules for its mathematical operators that mathematics does.
- The acronym **BODMAS** is a useful way to remember the order of operations:

- **Parentheses** have the highest precedence and can be used to force an expression to evaluate in the order you want. Since expressions in parentheses are evaluated first, $2*(3-1)$ is 4, and $(1+1)**(5-2)$ is 8. You can also use parentheses to make an expression easier to read, as in $(\text{minute}*100)/60$, even though it doesn't change the result.
- **Exponentiation** has the next highest precedence, so $2**1+1$ is 3 and not 4, and $3*1**3$ is 3 and not 27.
- **Multiplication** and **Division** have the same precedence, which is higher than **Addition** and **Subtraction**, which also have the same precedence.
- So $2*3-1$ yields 5 rather than 4, and $2/3-1$ is -1, not 1 (remember that in integer division, $2/3=0$).
- Operators with the same precedence are evaluated from left to right. If the `minute=59`, then in the expression `minute*100/60`, the multiplication happens first, yielding $5900/60$, which in turn yields 98. If the operations had been evaluated from right to left, the result would have been $59*1$ which is wrong. If in doubt, use parentheses.

Here is the order of precedence of the Python operators you have seen so far, from lowest to highest:

	Operator	Description
<i>lowest precedence</i>	Or	Boolean OR
	And	Boolean AND
	Not	Boolean NOT
	<code>=, !=, <, <=, >, >=, is, is not, in, not in</code>	comparisons, identity
		bitwise OR
	^	bitwise XOR
	&	bitwise AND
	<code><<, >></code>	bit shifts
	<code>+, -</code>	addition, subtraction
	<code>*, /, //, %</code>	multiplication, division, floor division, modulo
	<code>+x, -x, ~x</code>	unary positive, unary negation, bitwise negation
<i>highest precedence</i>	<code>**</code>	exponentiation

Operators at the top of the table have the lowest precedence, and those at the bottom of the table have the highest. Any operators in the same row of the table have equal precedence.

Control Flow: if, if-elif-else, for, while, break, continue, pass

If Statement

Decision making is required when we want to execute a code only if a certain condition is satisfied. The if, if...elif...else statement is used in Python for decision making.

Syntax:

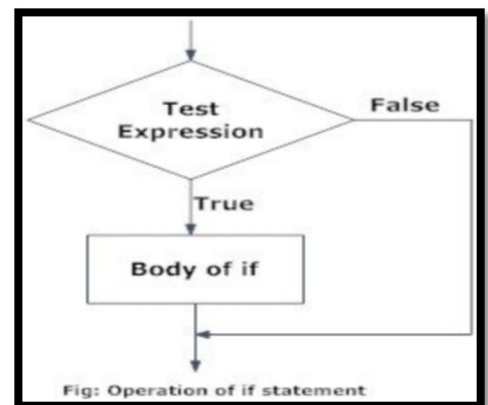
```
if test expression:  
    statement(s)
```

Here, the program evaluates the test expression and will execute statement(s) only if the test expression is True. If the test expression is False, the statement(s) is not executed. In Python, the body of the if statement is indicated by the indentation. The body starts with an indentation and the first unindented line marks the end. Python interprets non-zero values as True. None and 0 are interpreted as False.

Example: if Statement

```
# If the number is positive, we print an appropriate  
message
```

```
num = 3  
if num > 0:  
    print(num, "is a positive number.")  
print("This is always printed.")  
num = -1  
if num > 0:  
    print(num, "is a positive number.")  
print("This is also always printed.")
```



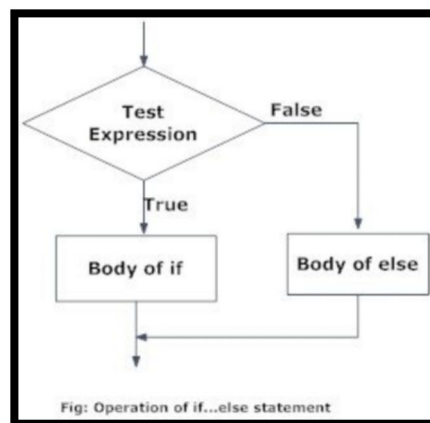
When you run the program, the output will be:

```
3 is a positive number  
This is always printed  
This is also always printed.
```

if...else Statement

Syntax:

```
if test expression:  
    Body of if  
else:  
    Body of else
```



The if..else statement evaluates test expression and will execute the body of if only when the test condition is True. If the condition is False, the body of else is executed. Indentation is used to separate the blocks.

Program checks if the number is positive or negative And displays an appropriate message

<pre>if num >= 0: print("Positive or Zero") else: print("Negative number")</pre>	<i>Output</i> Positive or Zero
---	---

if...elif...else Statement

Syntax:

if test expression:

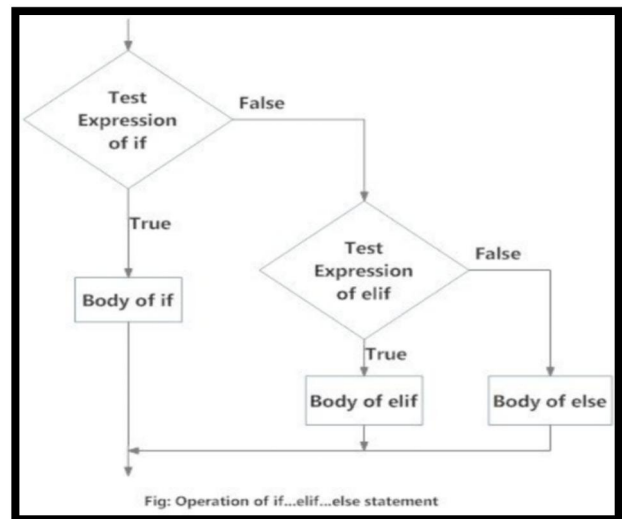
 Body of if

elif test expression:

 Body of elif

else:

 Body of else



The elif is short for else if. It allows us to check for multiple expressions. If the condition for if is False, it checks the condition of the next elif block and so on. If all the conditions are False, the body of else is executed. Only one block among the several if...elif...else blocks is executed according to the condition. The if block can have only one else block. But it can have multiple elif blocks.

program to check the number is positive or negative or zero and display an appropriate message

Try these two variations as well: num = 0, num = -4.5

num = 3.4

if num > 0:

 print("Positive number")

elif num == 0:

 print("Zero")

else:

 print("Negative number")

OUTPUT

Positive number

Nested if statements

We can have a if...elif...else statement inside another if...elif...else statement. This is called nesting in computer programming. Any number of these statements can be nested inside one another. Indentation is the only way to figure out the level of nesting. They can get confusing, so they must be avoided unless necessary.

Nested if Example

program to check if the number is positive or negative or zero and display an appropriate message with the help of nested if statement

```
num = float(input("Enter a number: "))
if num >= 0:
    if num == 0:
        print("Zero")
    else:
        print("Positive number")
else:
    print("Negative number")
```

Output 1

Enter a number: 5
Positive number

Output 2

Enter a number: -1
Negative number

Output 3

Enter a number: 0
Zero

FOR LOOP

The for loop in Python is used to iterate over a sequence (list, tuple, string) or other iterable objects.

Syntax:

```
for val in sequence:
    loop body
```

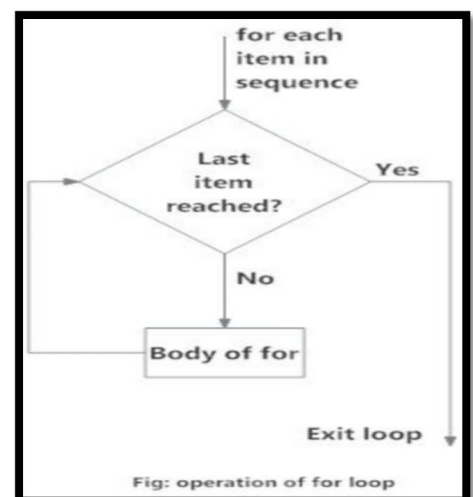
Here, val is the variable that takes the value of the item inside the sequence on each iteration.

Loop continues until we reach the last item in the sequence. The body of for loop is separated from the rest of the code using indentation.

Example: for Loop

program to find sum of all numbers stored in a list

```
numbers=[6,5,3,8,4,2,5,4,11]
sum=0
for val in numbers:
    sum=sum+val
```



```
print("the sum is",sum)
```

When you run the program, the output will be:

The sum is 48

The range() function

- We can generate a sequence of numbers using range() function. range(10) will generate numbers from 0 to 9 (10 numbers).
- We can also define the start, stop and step size as range(start, stop,step_size). step_size defaults to 1 if not provided.
- The range object is "lazy" in a sense because it doesn't generate every number that it "contains" when we create it.
- This function does not store all the values in memory. it would be inefficient. So it remembers the start, stop, step size and generates the next number on the go.
- To force this function to output all the items, we can use the function list().
- The following example will clarify this.

<pre>print(range(10)) print(list(range(10))) print(list(range(2,8))) print(list(range(2,20,3)))</pre>	<pre>OUTPUT range(0, 10) [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] [2, 3, 4, 5, 6, 7] [2, 5, 8, 11, 14, 17]</pre>
---	--

- We can use the range() function in for loops to iterate through a sequence of numbers.
- It can be combined with the len() function to iterate through a sequence using indexing.

#program to iterate through a list using **indexing**

<pre>Genre=['pop','rock','jazz'] For i in range(len(genre)): Print("I like", genre[i])</pre>	<pre>Output I like pop I like rock I like jazz</pre>
--	--

for loop with else

A for loop can have an optional else block as well. The else part is executed if the items in the sequence used in for loop exhausts. The break keyword can be used to stop a for loop. In such cases, the else part is ignored. Hence, a for loop's else part runs if no break occurs.

<pre>digits=[0,1,5] for i in digits: print(i) else: print("no items left")</pre>	<pre>OUTPUT 0 1 5 No items left.</pre>
--	--

- Here, the for loop prints items of the list until the loop exhausts.
- When the for loop exhausts, it executes the block of code in the else and prints No items left.
- This for...else statement can be used with the break keyword to run the else block only when the break keyword was not executed.

#program to display student's marks from record

```
student_name= 'soyuj'
marks={'james':90, 'jules':55, 'arthur':77}
for students in marks:
    if students==students_name:
        print(marks[student])
        break
else:
    print('no entry with name found')
```

Output

No entry with that name found.

While loop

The while loop in Python is used to iterate over a block of code as long as the test expression (condition) is true. We generally use this loop when we don't know the number of times to iterate beforehand.

Syntax :

```
while test_expression:
```

 Body of while

In the while loop, test expression is checked first. The body of the loop is entered only if the test_expression evaluates to True. After one iteration, the test expression is checked again. This process continues until the test_expression evaluates to False. In Python, the body of the while loop is determined through indentation. The body starts with indentation and the first unindented line marks the end. Python interprets any non-zero value as True. None and 0 are interpreted as False.

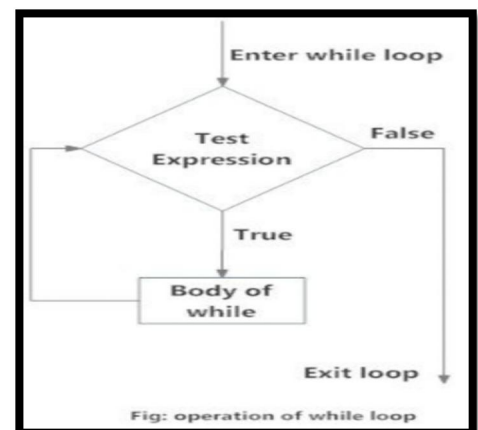
Example: program to add natural numbers upto n

```
n=int(input("enter n:"))
sum=0
i=1
while i<=n:
    sum=sum+i
    i=i+1
print("the sum is:", sum)
```

When you run the program, the *output* will be:

Enter n: 10

The sum is 55



While loop with else

Same as with for loops, while loops can also have an optional else block. The else part is executed if the condition in the while loop evaluates to False. The while loop can be terminated with a break statement. In such cases, the else part is ignored. Hence, a while loop's else part runs if no break occurs and the condition is false.

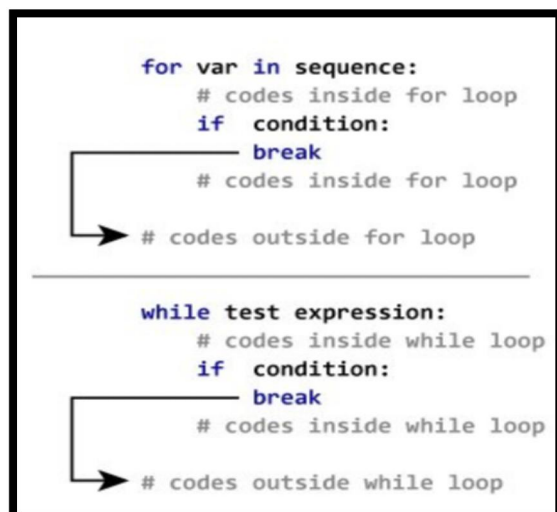
```
counter=0
while counter<3:
    print("inside loop")
    counter=counter+1
else:
    print("inside else")
```

Output
 Inside loop
 Inside loop
 Inside loop
 Inside else

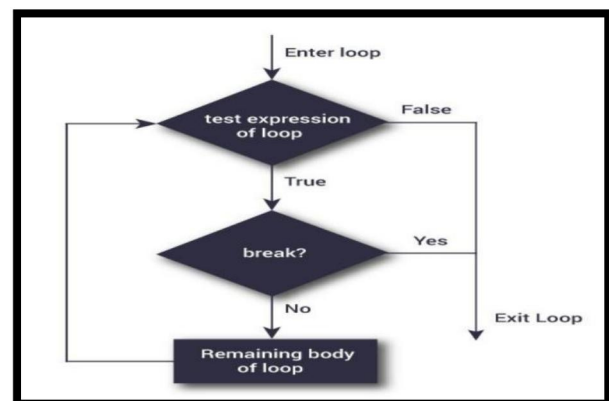
break statement

The break statement terminates the loop containing it. Control of the program flows to the statement immediately after the body of the loop. If the break statement is inside a nested loop (loop inside another loop), the break statement will terminate the innermost loop.

Working of the break statement



Flowchart of break statement



Example:

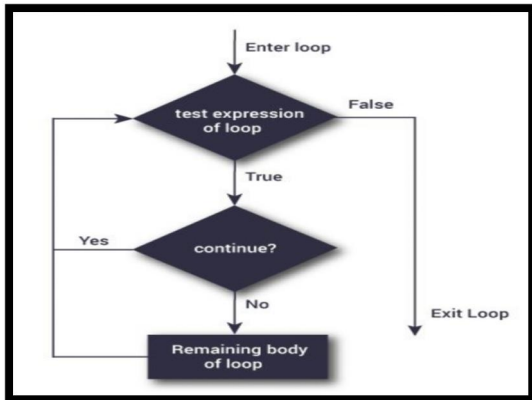
```
for val in "string":
    if val=="i":
        break
    print(val)
print("the end")
```

Output
 S
 T
 R
 The end

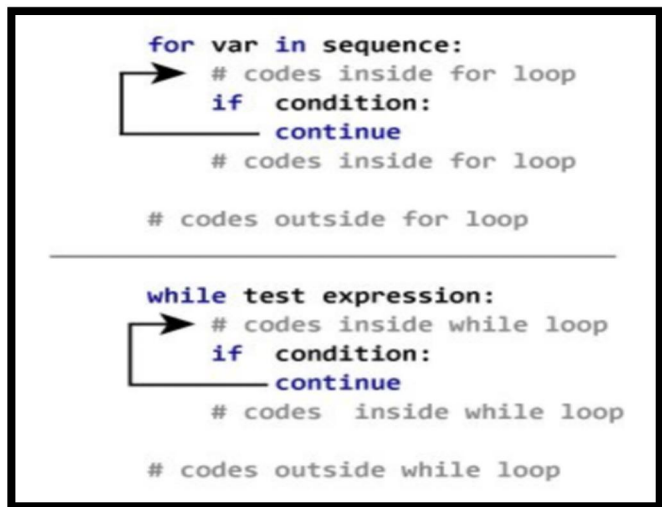
In this program, we iterate through the "string" sequence. We check if the letter is i, upon which we break from the loop. Hence, we see in our output that all the letters up till i gets printed. After that, the loop terminates.

Continue statement

The continue statement is used to skip the rest of the code inside a loop for the current iteration only. Loop does not terminate but continues on with the next iteration.



Flowchart of Continue statement



Working of the continue statement

Example:

```
for val in "string":
    if val == "i":
        continue
    print(val)
print("the end")
```

Output
S
T
R
N
G
The end

Pass Statement

In Python programming, the pass statement is a null statement. The difference between a comment and a pass statement is that while the interpreter ignores a comment entirely, pass is not ignored. However, nothing happens when the pass is executed. It results in no operation (NOP).

Suppose we have a loop or a function that is not implemented yet, but we want to implement it in the future. They cannot have an empty body. The interpreter would give an error. So, we use the pass statement to construct a body that does nothing.

Example:

```
Sequence={'p','a','s','s'}
For val in sequence:
    pass
```